

Assignment 1:

Pacman Programming

Name: Rishabh Ray
Student ID: 34525416

TABLE OF CONTENTS:

1. How I Started
2. Challenges and Solutions
 - a. Q1A: Single Food Dot Collection
 - b. Q1B: Nearest Food Dot Collection
 - c. Q1C: Maximizing Food Collection
 - d. Q2: Adversarial Search
3. What I Learned
4. Conclusion
5. References

How I Started

I started by carefully reading the problem descriptions for Q1A, Q1B, Q1C, and Q2. The goal was to use search algorithms to help Pac-Man gather food dots while dodging obstacles like walls and ghosts. Every subproblem had its own set of difficulties: *Berkeley AI materials*. (n.d.). Berkeley AI (CS188: Artificial Intelligence).

Q1A: Single food dot collection.

Q1B: Nearest food dot collection.

Q1C: Maximizing food collection.

Q2: Adversarial search against ghosts.

Examining the Codebase

I looked through the supplied Pac-Man files (game.py, search.py, etc.) to figure out how the game state, actions, and grid were represented, and I found several important elements:

- GameState Class: Symbolized the locations of the walls, ghosts, food grid, and Pac-Man.
- Actions Class: Specified movement directions (East, West, South, North and Stop) and successor computations.

Challenges and Solutions

Challenge: Understanding the getSuccessor function

Initially, I faced difficulty implementing GameState.getLegalActions correctly because the getSuccessor function did not recognize it.

Solution: To overcome this, I wrote custom logic to calculate legal actions by checking if moving in a specific direction would lead Pac-Man into a wall. This ensured that only valid moves were considered during search.

```
# -----
#defines the legal actions that can be made
actions = [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST, Directions.STOP]
successors = []

#loop to check for the next successor or position from the current position of the pacman
for action in actions:
    nextx, nexty = Actions.getSuccessor(state, action)
    #check whether there is a wall in the legal move
    nextx, nexty = int(nextx), int(nexty)
    if not self.walls[nextx][nexty]:
        nextstep = (nextx, nexty)
        stepcost = 1
        successors.append((nextstep, action, stepcost))
return successors
```

Q1A: Single Food Dot Collection

Approach

I implemented the A* search algorithm to guide Pac-Man to the nearest food dot using Manhattan distance as the heuristic. The pseudocode from Lecture 3a on informed search strategies helped structure my solution. I also referred to external resources like GeeksforGeeks and GitHub repositories for Python implementations of A*.

Berkeley_ai-pacman_projects/Project 1 Pacman/search.py at master · lzervos/berkeley_ai-pacman_projects. (n.d.). GitHub.

Pseudocode for A* Search

function UNIFORM-COST-SEARCH(problem) returns solution or failure

- Initialize the frontier (OPEN list) using the initial state of problem
- Loop
 1. if OPEN is empty then return failure
 2. remove the lowest-cost node from OPEN
 3. if the node is a goal then return corresponding solution.
 4. add the chosen node to the CLOSED list.
 5. expand the chosen node n, generating all successors
 6. for each successor node n':
 - a. compute a tentative estimate: $f'(n') = g(n) + c(n, n') + h(n)$
 - b. if n' is in CLOSED then discard it
 - c. else if n' is not in OPEN then add it to OPEN
 - d. else if n' is in OPEN with $f'(n) < f(n')$ then update the node and re-compute its priority
 7. END

```
def astar_heuristic(current, goal):  
    #calculated the heuristic distance from the current state to goal state  
    return util.manhattanDistance(current,goal)
```

Challenge: Failing to calculate the existence of a wall in the next x and y position

Solution: Converting of next x and y coordinates to int as a result of this calculation could potentially be a floating-point number due to the way vectors are defined

```
#loop to check for the next successor or position from t  
for action in actions:  
    nextx, nexty = Actions.getSuccessor(state,action)  
    #check whether there is a wall in the legal move  
    nextx, nexty = int(nextx), int(nexty)  
    if not self.walls[nextx][nexty]:  
        nextstep = (nextx, nexty)
```

Q1B: Nearest Food Dot Collection

Approach:

I started with the same problem and solver code that I earlier used in q1a and logically modified it to collect the dot by calculating the minimum of the Manhattan distance from

the multiple food dots present, I am sure there is a better way to minimize the node extensions as required by the questions but this approach worked for me and hence I applied it.

```
def astar_heuristic(current, goal):
    #calculated the heuristic distance from the current state to goal state
    # and select only the minimum one to return as goal state
    foodList = goal.asList()
    if not foodList:
        return 0
    return min(util.manhattanDistance(current, food) for food in foodList)
```

Challenge: Optimizing Node Expansions

While my implementation worked, it was not optimized for minimizing node expansions.

Solution

Although I did not fully optimize node expansions due to time constraints, I acknowledged that clustering or density-based heuristics could improve efficiency.

Q1C: Maximizing Food Collection

Solving the C part of Q1 took the most time for me. Initially, I started with the same code as Q1b to at least check the feasibility of my code, but that did not work out, possibly due to the need to store positions for all the food dots that need to be eaten. I took inspiration from Stack Overflow for this as well. *PacMan: What kinds of heuristics are mainly used?* (n.d.). Stack Overflow. *PACMAN: A short path for eating all the dots.* (n.d.). Stack Overflow.

Challenge: I faced an issue with storing the food dots, as my previous code was good for storing a single food dot.

Solution: To solve this, I made a food grid and stored the food location in a 2-D array with coordinates x,y, so that I can store the position of all the food dots.

I had to update my isGoalState function to return the count of the number of food dots remaining in the layout to account for multiple food dots.

Similarly successor function was updated to account for a 2D array of food dots.

```
def __init__(self, gameState: GameState):
    """
    Stores the start and goal.

    gameState: A GameState object (pacman.py)
    costFn: A function from a search state (tuple) to a non-negative number
    goal: A position in the gameState
    """
    self.startingLayout: GameState = gameState
    self.walls = gameState.getWalls()
    self.foodGrid = gameState.getFood() #stores using foodgrid as their are multiple food dots
    self.startPosition = gameState.getPacmanPosition() # getting the starting postion of the pacman
```

Challenge: The Foodgrid instance was providing a problem while using it in Q1c as it showed corrupt states during search.

Solution: After checking through YouTube videos on Pac-Man problems, using Generative AI to understand the errors in the code and GitHub, I realised the need to use Python's Deepcopy to ensure that each state's food grid was independent:

```
@log_function
def getStartState(self):
    """
    returns the starting position of the game with pacman location and a the locations of the food dots.
    """
    return (self.startPosition, self.foodGrid.deepCopy())
```

Challenge: Using A* algorithm with the proper heuristic metric to make the pacman go to the food where the density is higher.

Solution: In the custom heuristic defined by me I checked for closet food to the pacman and then find the cluster of food using the same logic to make the pacman go towards the largest cluster.

```
# Encourage Pac-Man to reach areas with the most food
return closest_food_distance + random_factor - 0.5 * max_food_cluster # Reward large clusters
```

Challenge: Pacman getting stuck in maps where 2 food dots are at the same distance

Solution: usage of random factor in q1c, where if the distance from 2 food dots is the same, I introduced a small difference by using the return closest_food_distance + random_factor - 0.5 * max_food_cluster clause. Where the pacman favoured one food dot over another, or even at the same distance

```
def heuristic(state, problem):
    """Heuristic function for A* search using food clustering strategy."""
    position, foodGrid = state
    foodList = foodGrid.asList()

    if not foodList:
        return 0 # No food left, heuristic cost is zero

    # Use nearest food distance as base heuristic
    closest_food_distance = min(util.manhattanDistance(position, food) for food in foodList)

    # This counts how many food dots are at this minimum distance
    closest_foods = [food for food in foodList if util.manhattanDistance(position, food) == closest_food_distance]

    # I realised the code getting stuck when the food dots were at the same distance so
    # this introduce a randomness when there are multiple closest food dots
    if len(closest_foods) > 1:
        x, y = position
        random_factor = ((x ^ y) % 5) * 0.1 # Generates values between 0 and 0.4
    else:
        random_factor = 0

    # Find the food cluster with the most dots
    max_food_cluster = 0
    for food in foodList:
        cluster_size = sum(1 for f in foodList if util.manhattanDistance(food, f) <= 3)
        max_food_cluster = max(max_food_cluster, cluster_size)

    # Encourage Pac-Man to reach areas with the most food
    return closest_food_distance + random_factor - 0.5 * max_food_cluster # Reward large clusters
```

Q2 Adversarial Search:

For this, I used the concept of the Minimax algorithm with Alpha-Beta Pruning as required by the code, where Pac-Man acted as the maximizing agent, and ghosts acted as minimizing agents.

Pseudo code from the class lectures given below:

```
function ALPHA-BETA-SEARCH(state) returns an action
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in ACTIONS(state) with value v

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \geq \beta$  then return v       $\beta$  cut-off
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return v

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for each a in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return v       $\alpha$  cut-off
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return v
```

I went through the class notes on adversarial search, where I got the pseudo code for running the minimax algorithm with pruning, which I then applied by getting to know that agent 0 is always the pacman and the rest are the Ghosts.

Alpha-Beta Pruning was used to improve efficiency by skipping branches that couldn't affect the final decision:

```
alpha = max(alpha, bestValue)
if beta <= alpha: #this prunes the path
    break
```

Alpha: The best value Pac-Man can guarantee at the current level.

Beta: The best value ghosts can guarantee at the current level.

Challenge: Oscillation of Pac-Man in corners when the food dot is nearby

Solution 1: Initially, I used a heuristic to change the movement of Pac-Man when facing a ghost, meeting a corner or going near the food dots.

Later, I removed the custom heuristic class and used the given class to introduce a heuristic for Pac-Man to go towards the nearest food dot by giving a bonus on every step. This approach worked to avoid most of the cases where the pacman was getting stuck in the corner, even while food was nearby.

Solution2: To prevent Pac-Man from getting stuck in repetitive movements (e.g., moving back and forth between two positions), I also implemented a oscillation detection mechanism which Analyzed recent moves stored in action_history and detected North-South or East-West loops. And then Forced Pac-Man to choose alternative actions when oscillations were detected.

What I learned while completing the Pac-Man Problem:

1. How to implement A* search algorithm:
 - a. Learned how A* combines cost-to-reach ($g(n)$) and heuristic ($h(n)$) to guide search efficiently.
 - b. Designed problem-specific heuristics:
 - c. Used Manhattan distance for Q1A.
 - d. Developed a clustering-based heuristic for Q1C to prioritize dense food areas.
 - e. Understood the importance of state representation (e.g., position + food grid) and used `deepcopy()` to prevent state corruption.
2. Reading and Editing Code
 - a. Gained experience navigating and modifying existing codebases like `GameState` and `Actions`.
 - b. Debugged issues with `getSuccessor` and implemented custom logic for legal actions.
3. Adversarial Search
 - a. Implemented Minimax with Alpha-Beta Pruning for Q2, where Pac-Man maximized its score, and ghosts minimized it.
 - b. Improved efficiency by pruning unnecessary branches and handling multi-agent systems.

Conclusion

This assignment taught me how to implement A*, Minimax, and Alpha-Beta Pruning while balancing competing objectives like food collection and ghost avoidance. It also improved my skills in debugging, heuristic design, and adapting existing codebases, which will be invaluable in future AI projects.

References:

1. https://ai.berkeley.edu/project_overview.html
2. https://github.com/lzervos/Berkeley_AI-Pacman_Projects/blob/master/Project%201%20Pacman/search.py
3. <https://stackoverflow.com/questions/9994913/pacman-what-kinds-of-heuristics-are-mainly-used>
4. <https://stackoverflow.com/questions/12877560/pacman-a-short-path-for-eating-all-the-dots>
5. Generative AI has been used with guidance for the assignment.